

A
Project Report
ON
RUMOUR DETECTION SYSTEM

submitted in partial fulfillment of the requirements
for the award of the Degree of
MASTER OF SCIENCE
IN
INFORMATION TECHNOLOGY



SUPERVISOR:

Mr. Dipanshu Sharma
Assistant Professor

SUBMITTED BY:

Abhishek [2416090001]
Km. Neha [2416090002]
Vishu Katariya [2416090003]
[M.Sc. (IT) IV Sem]

Faculty of Computer Science & Information Technology
Motherhood University, Roorkee (UK)

MAY 2026

CERTIFICATE

This is to certify that Mr. **Abhishek**, Miss. **Neha** and Mr.**Vishu Katariya**, student of the **Master of Science in Information Technology** 4th semester, have completed and submitted their project report entitled “**Rumour Detection System**” under our guidance and supervision.

The project work carried out by them is a genuine and original contribution, completed during the academic session as a part of the curriculum requirements for the award of the degree. The students have demonstrated sincere effort, dedication, and a clear understanding of the subject matter while developing this project. To the best of our knowledge, this work has not been submitted to any other university or institution for the award of any degree or diploma. The project meets the required academic standards and is approved for submission.

Mr. Dipanshu Sharma

Assistant Professor

CANDIDATE'S DECLARATION

We hereby declare that this project report titled “Rumour Detection System” is prepared for academic submission in accordance with the departmental format. The report is based on the supplied software package and on my understanding of the source code, interfaces, workflows, and data handling mechanisms included in that package.

We further declare that the content of this report is prepared for educational use. Wherever assumptions were necessary because supporting datasets or measured performance logs were not included in the ZIP archive, those assumptions have been clearly identified in the relevant chapters.

Abhishek [2416090001]

Km. Neha [2416090002]

Vishu Katariya [2416090003]

[M.Sc. (IT) IV Sem]

ACKNOWLEDGEMENT

We express sincere gratitude to **Prof Dr. Ravindra Kumar** (Dean & HOD), Department of Computer Science and Information Technology, **Motherhood University, Roorkee**, for providing us the opportunity and necessary facilities to complete this project.

We are thankful to **Mr. Dipanshu Sharma** Assistant Professor, Department of Computer Science and Information Technology, who helped us complete this project successfully by providing guidance, encouragement, and valuable suggestions throughout our project. We also thank all faculty members of cs/it and others who helped directly or indirectly in completing this project successfully.

We also acknowledge the role of open source Python libraries such as Streamlit, scikit-learn, feedparser, newspaper3k, Plotly, SpeechRecognition, langdetect, and deep-translator, all of which contribute to the construction of the analysed system. The report presented here attempts to convert the available implementation into a complete academic description suitable for submission.

ABSTRACT

The Rumour Detection System is a Python based web application implemented with Streamlit and supported by machine learning, text preprocessing, live news evidence collection, and simple database persistence. Although the title of the project uses the phrase “RUMOUR Detection System”, the inspected implementation actually behaves like a news credibility and evidence aggregation tool. It accepts a headline, article text, voice based text input, or a URL. The system then detects language, translates non-English input into English, cleans the text, obtains a machine learning prediction using a TF-IDF vectorizer and logistic regression classifier, and collects corroborating evidence from trusted RSS feeds, Google News RSS, and selected official domains such as PIB, RBI, WHO, and UGC. The final result shown to the user is intentionally evidence-first rather than model-only.

This report studies the source package, documents each module, explains the algorithms and control flow, analyses the SQLite schema, and presents likely testing scenarios and deployment steps. Since the ZIP archive does not include the training dataset or experimentally logged accuracy reports, the document avoids unsupported numerical claims and instead focuses on architecture, implementation quality, functionality, limitations, and future improvement opportunities. The outcome is a submission-ready Computer Science project report prepared according to the uploaded departmental template.

TABLE OF CONTENTS

Certificate	I
Candidate's declaration	II
Acknowledgement	III
Abstract	IV
Chapter 1: Introduction	1-3
1.1 Background of the problem	1-2
1.2 Objectives of the project	2-2
1.3 Scope of the project	2-3
1.4 Need for the system	3-3
Chapter 2: Literature Survey	4-7
2.1 Text classification in natural language processing	4-4
2.2 Misinformation and credibility analysis systems	5-5
2.3 Multilingual text handling and translation	5-5
2.4 Explain ability and evidence-first user interfaces	5-6
2.5 User interface frameworks for ML prototypes	6-6
2.6 Comparative discussion	6-7
2.7 Comparative table of related approaches	7-7
Chapter 3: System Analysis and Design	8-14
3.1 Folder and file structure	8-9
3.2 Overall architecture	9-10
3.3 Workflow analysis	10-11
3.4 Input design	11-11
3.5 Output design	11-12
3.6 Functional module description	12-12
3.7 Data flow and control flow	13-13
3.8 Dataflow diagram (DFD)	13-14

Chapter 4: Implementation Details	15-29
4.1 Main application file	15-20
4.2 User interface package	20-21
4.3 News search and evidence collection	21-25
4.4 Stance detection heuristics	25-25
4.5 Training pipeline	26-28
4.6 Dependency analysis	28-28
4.7 Security and robustness observations	28-29
 Chapter 5: Testing, Observations and Discussion	 30-33
5.1 Observed package detail	30-30
5.2 Representative functional test cases	30-31
5.3 Strengths of the current implementation	31-31
5.4 Limitations and gaps	31-32
5.5 Performance discussion without unsupported metrics	32-32
5.6 Deployment and execution steps	32-33
 Chapter 6: Conclusion and Future Scope	 34-35
6.1 Future scope	34-35
 References	 36-36
 Appendix:	 VII-XI
Appendix A: Screenshots of the application	VII-XI
Appendix B: Execution Commands	XI-XI

CHAPTER - 1

Introduction

Rumour Detection System is presented in the submitted package as a compact Python application that combines a machine learning classifier with live source verification. From an academic perspective, the project belongs to the broad domain of natural language processing, information retrieval, and decision support systems. The inspected code shows that the application is designed for practical use through a browser-based interface built with Streamlit. **The system allows the user to analyse four kinds of input: a headline, manually typed text, text obtained from voice input, and article text extracted from a pasted URL.** The main intention is to classify input content and then support that classification with external evidence.

A careful scan of the source files shows an important conceptual distinction. The software title uses the phrase “Rumour Detection System”, yet the actual application logic is closer to a fact-check assistance or news authenticity analysis platform. This is not a criticism; rather, it is an observation that the implemented features do not focus on rumour recognition such as satire, sarcasm, punchline detection, or sentiment-based comic interpretation. Instead, the code attempts to determine whether content is likely real, fake, misleading, or unclear by combining a trained logistic regression model with evidence from trusted publishers and official sources.

In many student projects, machine learning output is shown as the final answer. This project takes a more cautious route. The function ``final_label_merge`` deliberately treats the model prediction as secondary and gives more importance to official confirmations, trusted contradiction signals, and an aggregate evidence score. That design choice is meaningful because it reduces over-reliance on a single classifier and encourages manual verification through links displayed in the user interface.

1.1 Background of the problem

The modern information ecosystem is overwhelmed by fast-moving headlines, edited screenshots, unverified claims, and recycled news fragments. Users often encounter

information first on social media or messaging platforms, where context is limited and fact-checking is delayed. In such situations, an assistant system that can quickly inspect text, compare it against known publishers, and search official websites becomes useful. Even when a system cannot guarantee truth, it can provide a structured starting point for human judgement.

Traditional misinformation detection methods rely either on manual fact-checking or on machine learning models trained on labelled articles. Manual review is accurate but slow. Pure machine learning can be fast but may fail on emerging events, multilingual content, or domain-specific claims. **The submitted project tries to balance these issues by using both prediction and external evidence.** Language detection and translation allow the system to accept broader user input. RSS feeds provide timely publication data from major news organisations. Official domain lookup helps when claims refer to government bodies or institutional authorities. Together, these components turn the application into a lightweight decision-support workflow rather than a black-box classifier.

The project is educationally relevant because it demonstrates how web technologies, natural language processing, machine learning, and database persistence can be combined inside one deployable software artifact. It also shows how user experience considerations such as colour themes, result gauges, expanders, metrics, and warnings can improve the trustworthiness of technical output.

1.2 Objectives of the project

- To analyse the supplied ZIP package and convert its implementation into a formal project report in Computer Science format.
- To explain the architecture of a Streamlit based application that accepts headline, article text, voice, and URL inputs.
- To document how machine learning, live RSS feeds, Google News search, and official domain checks are integrated.

1.3 Scope of the project

The scope of the project includes source code review, functional interpretation, user interface analysis, dependency mapping, database schema inspection, and preparation of a submission-

ready report. It does not include retraining the classifier because the training datasets expected by `train_model.py` are absent from the uploaded package. It also does not include measured runtime benchmarks or final accuracy values, because such evidence is not present in the provided files. Nevertheless, the source package is sufficient to reconstruct the intended workflow and to present a complete academic description of the software.

At the application level, the system scope covers English and non-English textual input, simple machine learning classification, evidence gathering from predefined sources, and basic user history display. It does not include deep semantic reasoning, image forensics, social media network analysis, user role management, or authenticated multi-user dashboards. The SQLite database stores only simple user and history information and is therefore appropriate for a prototype or academic demonstration rather than a production-scale deployment.

1.4 Need for the system

A system of this type is useful because users require rapid triage when reading suspicious or sensational content. A prototype that shows both a predictive result and supporting links can reduce blind trust in forwarded messages. The warning messages built into the result view show a healthy design attitude: the application repeatedly tells the user not to rely solely on the model output and to verify links manually. Such transparency is valuable in classroom projects because it demonstrates responsible AI practice.

The system is also useful as a learning platform. Students can observe how multiple libraries interact in a full-stack Python application. They can trace how the input is obtained, how it is transformed by helper utilities, how the model is loaded from serialized files, how web feeds are parsed, how scores are merged, and how the final answer is rendered in a modern dashboard. This makes the project a good example of applied integration rather than algorithmic theory alone.

CHAPTER - 2

Literature Survey

Literature survey in software projects serves two purposes. First, it places the project within an existing body of techniques. Second, it shows why a particular design is suitable for the selected problem. Since the submitted package combines machine learning, feed retrieval, translation, and rule-based decision fusion, the most relevant literature themes include misinformation detection, text classification using TF-IDF and logistic regression, evidence-based fact-check assistance, multilingual preprocessing, and user-facing explainable AI.

A practical student implementation often borrows components from several research traditions rather than following one pure model. The inspected project reflects exactly that pattern. Its classifier resembles classical document classification pipelines, its source checking resembles lightweight fact retrieval, and its final score fusion resembles heuristic decision support. Therefore, the literature survey below is organised around concepts rather than around a single algorithm family.

2.1 Text classification in natural language processing

Text classification is a foundational NLP task in which a textual input is assigned to one of several labels. Classical approaches convert documents into vector representations and then train a statistical classifier. Among these methods, TF-IDF remains one of the most interpretable feature extraction techniques. It transforms text into a sparse vector where each dimension reflects the importance of a term relative to document frequency. Logistic regression, support vector machines, and naive Bayes are commonly used with such features because they are fast, transparent, and effective for many medium-scale datasets.

The training script inside the submitted package follows this established design. It reads two CSV files labelled as fake and true, concatenates title and text fields where available, cleans the content, vectorizes it using `TfidfVectorizer(max_features=5000)`, and trains `LogisticRegression(max_iter=200)`. The choice is academically reasonable because it offers a good trade-off between implementation simplicity and practical performance.

2.2 Misinformation and credibility analysis systems

Misinformation analysis systems generally combine textual clues with external evidence. Text-only classification can identify patterns associated with sensational language, repetition, and emotionally charged claims, but it may fail when false and true texts share stylistic features. For this reason, many fact-check systems extend beyond document content and collect signals from publishers, historical claims, or known institutions. The current project reflects that line of thinking by querying trusted RSS feeds, Google News RSS, and official websites when certain authority terms are detected.

The strongest academic value of such systems lies in their acknowledgement that credibility is not purely linguistic. Timeliness, source authority, and corroboration all matter. The function ``build_conflict`` inside the project attempts to quantify whether evidence items support or contradict the claim. This resembles practical credibility aggregation rather than end-to-end fact verification, but it is still useful because it teaches students how to merge independent signals.

2.3 Multilingual text handling and translation

Real-world users often type in languages other than English or in mixed-language forms. The project addresses this issue by using ``langdetect`` for language identification and ``deep_translator.GoogleTranslator`` to translate non-English text into English. In multilingual NLP, translation into a common pivot language is a common engineering strategy when the downstream model is trained only on English data. The approach is not perfect, because translation may alter nuance or named entities, but it substantially broadens usability without requiring multiple language-specific classifiers.

From a literature perspective, multilingual preprocessing introduces trade-offs between inclusivity and fidelity. The submitted project accepts those trade-offs in favour of practical accessibility. In an academic report, this is worth noting because it reflects a real deployment decision rather than a purely theoretical exercise.

2.4 Explain ability and evidence-first user interfaces

Explainable AI is increasingly important when systems generate socially sensitive outputs. Instead of presenting a bare label, trustworthy systems explain why a result was produced, how

confident the underlying model is, and what evidence influenced the final decision. The result view in the scanned package aligns with this principle. It displays a final label, an evidence-strength gauge, an official-check indicator, explanatory bullets, and expandable sections for model details and evidence breakdown.

This design can be compared with explainable dashboards in health, finance, and misinformation contexts, where the user must understand the basis of a recommendation before acting. The project's repeated warning banners are especially notable because they explicitly discourage blind acceptance of the AI output. Such warnings support the broader literature on human-in-the-loop decision systems.

2.5 User interface frameworks for ML prototypes

Frameworks like Streamlit have become common in educational machine learning projects because they reduce front-end complexity while allowing rich interactive behaviour. With a small amount of Python code, students can deploy forms, buttons, charts, warnings, expanders, and data tables. In this project, Streamlit is used not just for layout but also for state management, theme application, and result persistence during a session. Modules under the `UI` package separate concerns into focused rendering functions, which makes the code easier to maintain than a monolithic single file.

The project also uses Plotly for gauge and bar charts. This combination of Streamlit plus Plotly has become a standard pattern for lightweight analytical dashboards. Therefore, the software reflects contemporary practice for rapid prototyping in data-driven applications.

2.6 Comparative discussion

When we compare classical ML pipelines, heuristic evidence fusion, and modern deep models, each has a different trade-off profile. Classical methods are transparent and resource efficient, deep models are context aware but expensive, and heuristic systems are flexible but difficult to calibrate. The submitted package chooses a hybrid route: a classical model provides the first signal, while heuristics and external sources refine the final answer. For a student project with limited infrastructure, this is an acceptable and pedagogically rich choice.

A limitation of this approach is that the heuristics are domain-specific and may not generalize well. Another is that RSS based evidence depends on available publications and network

connectivity. Nonetheless, from an educational perspective, the architecture is easier to explain and defend than a large opaque model with no transparency.

2.7 Comparative table of related approaches

Approach	Core idea	Strengths	Weaknesses	Relation to scanned project
TF-IDF + Logistic Regression	Sparse vector representation with linear classifier	Fast, interpretable, easy to train	Limited deep context understanding	Directly used in train_model.py and serialized model files.
Transformer based classifier	Contextual embeddings with deep attention layers	High semantic performance	Heavy resources and deployment complexity	Not used, but could replace current model in future versions.
Rule-based evidence scoring	Decision from handcrafted criteria	Transparent and adjustable	Can be brittle on unseen cases	Used in conflict building and evidence-strength scoring.
Fact-check retrieval systems	Search for supporting or contradictory articles	Useful for emerging claims	Depends on source availability	Implemented through RSS and Google News searches.
Official source verification	Look for institutional confirmation	High authority for governance and public notices	Coverage limited to specific domains	Implemented for PIB, RBI, WHO, UGC and generic gov domains.

Table 2.1 Comparison of related techniques and their relation to the scanned package

CHAPTER - 3

System Analysis and Design

System analysis begins by identifying the major components visible in the ZIP archive and then mapping how they interact when a user submits input. The project package follows a straightforward modular structure. `app.py` acts as the entry point and assembles the user interface. The `UI` package holds dedicated rendering functions for each input mode and result display. The `utils` package contains the actual analytical logic, including text cleaning, feed search, translation, scoring, and stance rules. The `database` package stores helper functions for SQLite operations, while the `model` folder stores serialized machine learning artifacts.

This chapter translates that implementation into architectural documentation. The goal is not merely to repeat filenames but to explain the rationale of the design and the flow of information from start to finish.

3.1 Folder and file structure

Path	Purpose
app.py	Main Streamlit entry point, model loading, session state, input flow, and orchestration.
UI/home.py	Home selector, logos, theme-dependent backgrounds, and mode switching.
UI/text_view.py	Text input form, voice input hook, and analysis trigger.
UI/url_view.py	URL extraction workflow and article preview.
UI/headline_view.py	Headline input mode.
UI/result_view.py	Metrics, gauges, evidence display, and history update.
UI/history_view.py	Recent searches and result distribution chart.
utils/analysis_engine.py	Language detection, translation, model prediction, RSS/official checks, final merge.
utils/news_search.py	Trusted RSS search, Google News RSS query, domain detection.
utils/scoring.py	Conflict construction, evidence score, and verdict fusion.
utils/text_utils.py	Text cleaning, normalization, domain extraction, deduplication.

utils/stance.py	Rule-based stance detection from titles/snippets.
database/db.py	SQLite initialization, save, and load helpers.
train_model.py	Training pipeline for TF-IDF and logistic regression model.
model/model.pkl	Serialized logistic regression classifier.
model/tfidf.pkl	Serialized TF-IDF vectorizer with 5000 features.
users.db	SQLite file observed with users and history tables.

Table 3.1 Folder and file mapping of the submitted ZIP package

3.2 Overall architecture

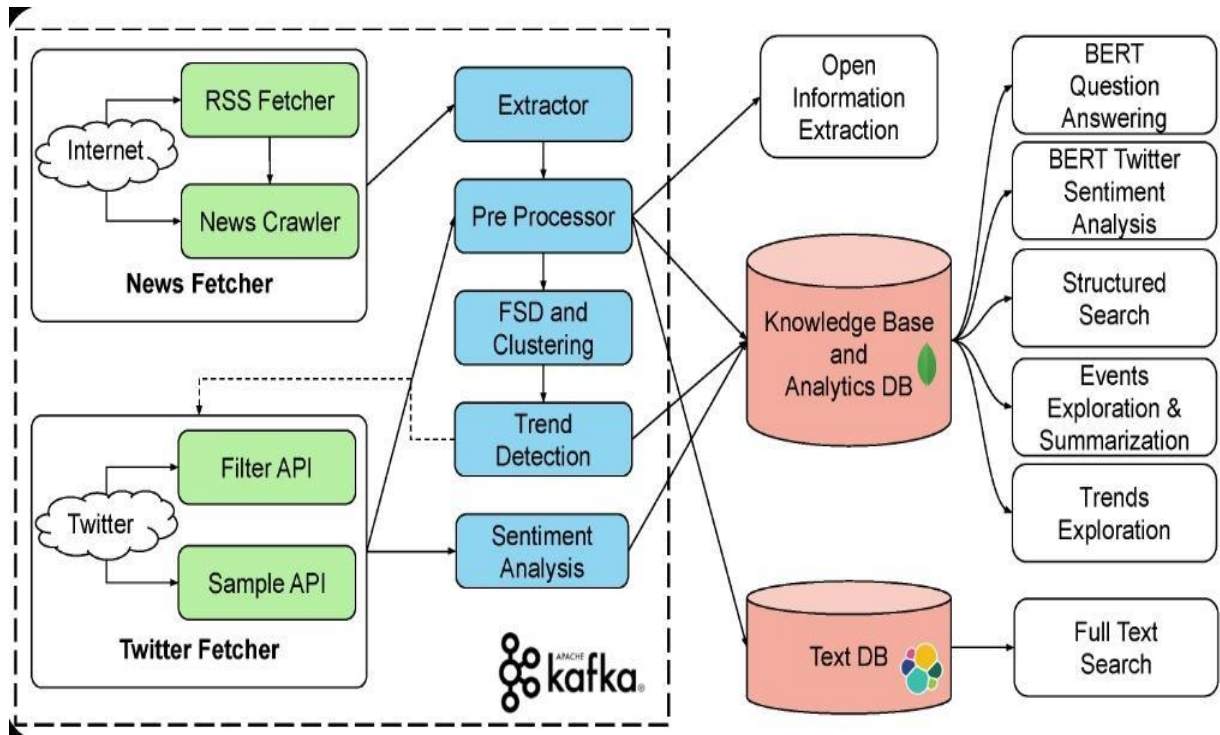


Fig. 3.1 System Architecture of the Rumour Detection System

The architecture can be described in six layers: interface layer, preprocessing layer, machine learning layer, evidence retrieval layer, scoring and fusion layer, and output layer. The interface layer collects user input through dedicated views for headline, text, and URL. The preprocessing layer detects language, translates when necessary, and cleans text into a model-ready form. The machine learning layer loads a logistic regression model and a TF-IDF

vectorizer from pickle files. The evidence retrieval layer performs RSS parsing and official-domain based checks. The scoring layer combines support, contradiction, and official-source signals into a final label. The output layer shows verdict cards, charts, explanations, evidence links, and warnings.

The architecture is appropriate for modular enhancement. A developer could replace the classifier, extend trusted sources, improve translation, or swap the database backend without rewriting the complete application. This modularity is one of the strengths visible in the scanned package.

3.3 Workflow analysis

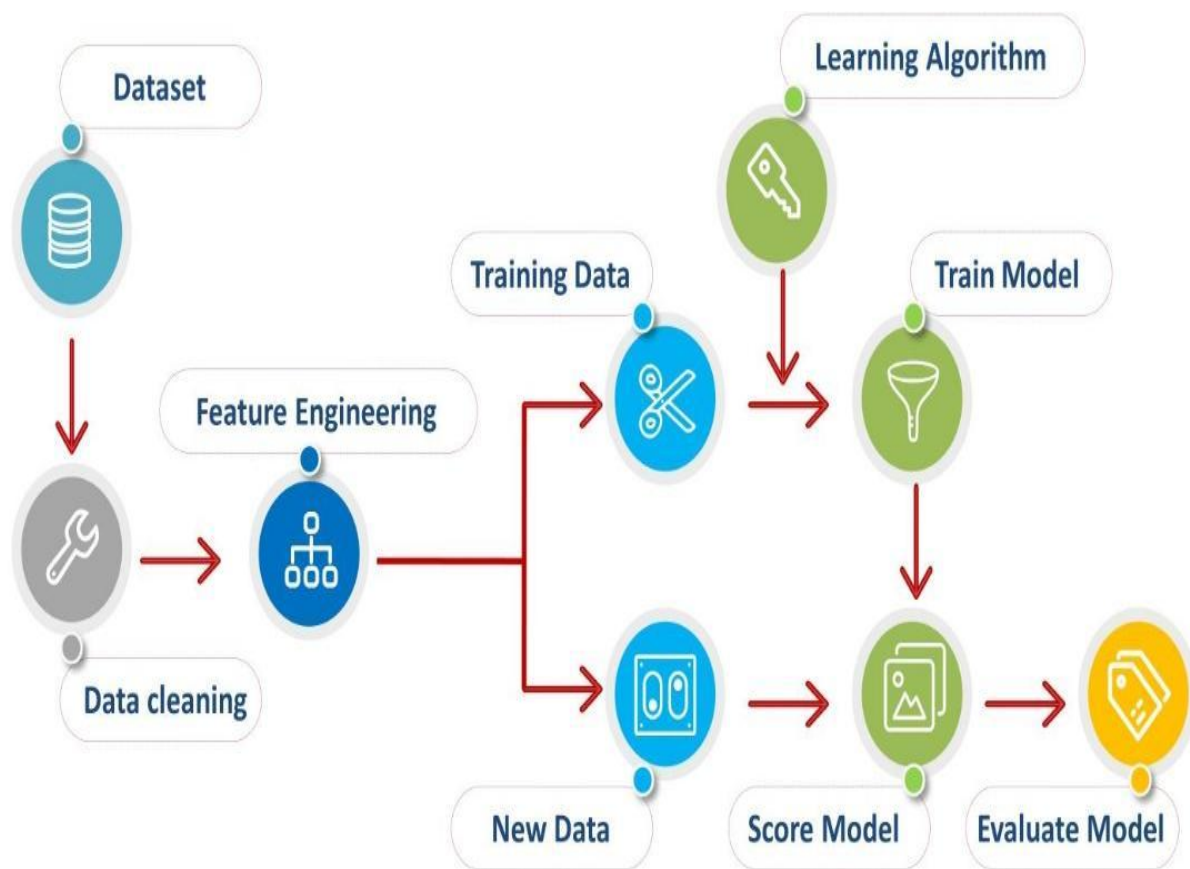


Fig. 3.2 Application Workflow

When a user begins on the home page, the application first asks what type of content will be analysed. This creates a simple decision gateway that keeps the interface uncluttered. If the user selects headline mode, the system accepts a short textual title. If text mode is selected, the

user can either type content or record voice input through the microphone. If URL mode is selected, the system attempts to extract article text before analysis. Regardless of entry route, the processing pipeline eventually converges to the same ``analyze_news`` function.

Inside ``analyze_news``, several sequential operations occur. The input is stripped and validated. Language is detected using a protected helper that returns English on failure. Translation occurs only when the detected language is not English. The cleaned translated text is then vectorized and passed into the classifier. After the ML output is obtained, search keywords are formed from the beginning of the translated input. These keywords are used to search trusted RSS feeds and Google News RSS. If terms such as UGC, RBI, WHO, CDC, PIB, or government are present, the system also identifies likely official domains and checks them separately. Finally, conflict scores and evidence strength are calculated, and the final verdict is produced.

3.4 Input design

The input design aims for ease of use rather than strict formality. A single application supports multiple interaction styles, which broadens the target user base. Headline mode is suitable when only a top-line claim is available. Text mode allows more detailed input and supports voice capture, making the system accessible to users who prefer speaking over typing. URL mode addresses a common user scenario: receiving a link and wanting the system to fetch the article automatically. This flexibility is a practical design advantage because misinformation often appears in different forms.

Input validation is intentionally lightweight. Empty inputs are rejected with warnings, and voice failures produce a warning instead of a crash. URL extraction failures are also surfaced to the user. The design choice favours continuity of use. Rather than exposing stack traces, the system provides concise messages that are understandable to non-technical users.

3.5 Output design

The output design is more sophisticated than the input forms. The result page provides three top metrics: final label, evidence strength, and official check. It then highlights the overall verdict using success, error, or warning styling based on whether the content appears real, fake, or unclear. A gauge chart visualizes evidence strength, while a secondary expander reveals

machine learning confidence and a separate model confidence gauge. Additional sections list official-site matches, trusted RSS evidence, and general Google News results. Finally, a breakdown table can be expanded for stance analysis, and a prominent caution reminds the user to verify information manually.

This output strategy serves two design goals. First, it maximizes transparency by exposing intermediate evidence. Second, it reduces cognitive overload by prioritizing a clear top-line answer and hiding secondary detail behind expanders. The result is suitable for educational demonstrations because a teacher or evaluator can see both user-friendly summary and technical detail.

3.6 Functional module description

Module	Key function(s)	Role in system
Model loader	load_model()	Loads pickle artifacts once using Streamlit cache.
Voice input helper	get_voice_input()	Uses microphone and Google recognition for speech to text.
Core analyzer	analyze_news()	_safe_detect_lang(), _safe_translate_to_en(), _ml_predict()
Search layer	search_trusted_rss(), google_news_rss_search(), search_official_sites_via_google_news()	Collects evidence from publishers and official domains.
Scoring layer	build_conflict(), compute_evidence_strength(), final_label_merge()	Creates support/contradiction summary and final verdict.
Result renderer	render_analysis_result()	Displays cards, charts, bullets, evidence, and warnings.
History renderer	render_history()	Visualizes recent prediction distribution.

Table 3.2 Functional modules and responsibilities

3.7 Data flow and control flow

From a data-flow perspective, the application transforms the original user input through a sequence of representations. Raw input becomes normalized text. Normalized text becomes cleaned feature text. Cleaned text becomes a sparse TF-IDF vector. The vector becomes a machine learning label and confidence. In parallel, a keyword string becomes multiple search queries that return evidence item dictionaries. Those dictionaries then become counts and scores. The final output is therefore not a single computation but a fusion of model signal, evidence retrieval, and heuristic interpretation.

From a control-flow perspective, Streamlit reruns the script whenever the page state changes. The design handles this by storing key values in `st.session\_state`, such as history, headline input, extracted text, and current mode. This is an important implementation insight because, in Streamlit, apparent page transitions are actually controlled by rerun-friendly state variables. The package uses this mechanism effectively.

3.8 Dataflow diagram (DFD)

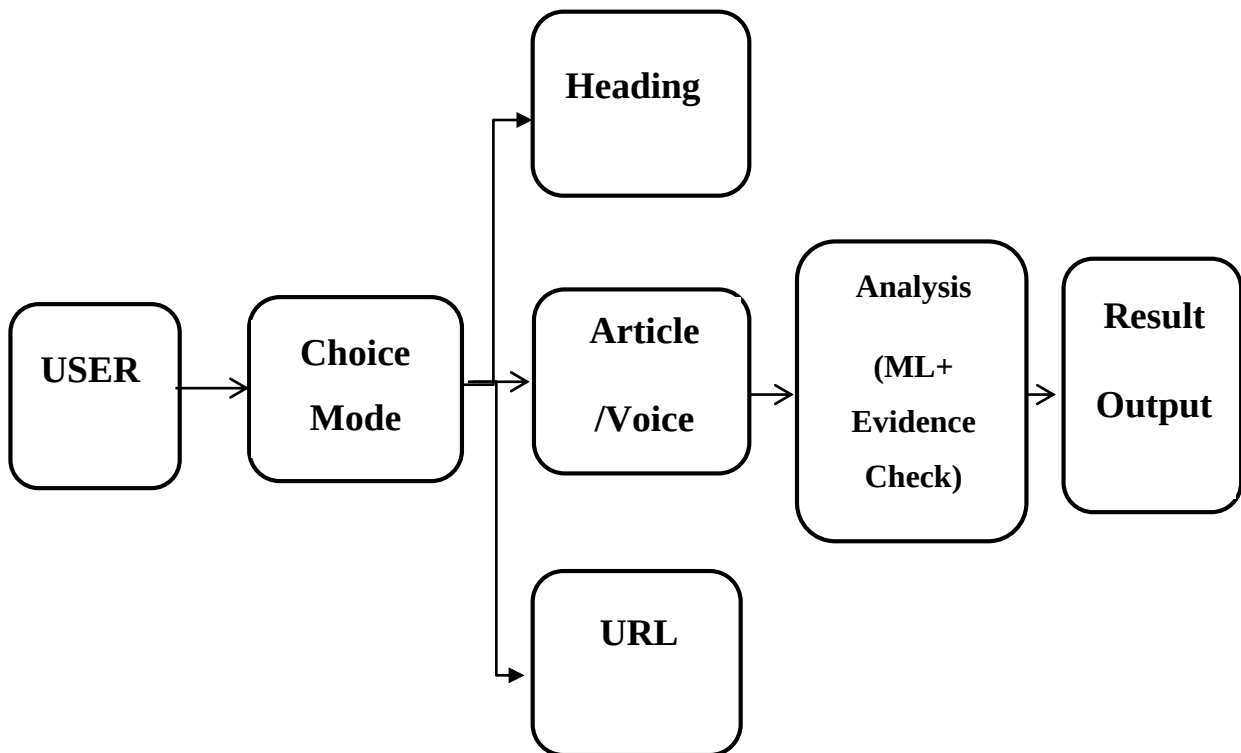


Fig. 3.3 Dataflow diagram based on observed contents

The project includes two SQLite-related artifacts. The file `database/db.py` defines a `history` table under a path `data/news.db`, whereas the supplied package also contains an actual `users.db` file with tables named `users` and `history`. The observed `users` table contains a username and hashed password field, while the observed `history` table stores username, text, and result. This suggests that authentication functionality was either removed from the current Streamlit interface or handled in an earlier version of the project. In the current visible application flow, the session history is maintained inside Streamlit session state rather than through direct database writes.

This discrepancy is academically useful to discuss because it reflects a common software engineering reality: project packages evolve over time, and not every file remains fully synchronized with the current front-end. The report therefore documents both the intended database helper functions and the actual database file that was present in the ZIP archive.

CHAPTER - 4

Implementation Details

This chapter converts the code scan into an implementation narrative. Rather than listing syntax alone, it explains how each file contributes to the final application. It also highlights design quality, dependency management, and noteworthy engineering decisions such as caching, broad exception handling, and use of serialized model artifacts.

4.1 Main application file

`app.py` is the central entry point. It imports the theme layer, home selector, rendering helpers, analytical utilities, and configuration values.

App.py code

```
# =====  
  
# Rumour Detection System - Streamlit App  
  
# =====  
  
import streamlit as st  
  
import pickle  
  
import speech_recognition as sr  
  
from UI.theme import apply_theme  
  
from UI.home import render_home_selector, render_back_button  
  
from UI.components import render_bullets_bold  
  
from UI.result_view import render_analysis_result  
  
from UI.history_view import render_history  
  
from UI.headline_view import render_headline_view  
  
from UI.text_view import render_text_view  
  
from UI.url_view import render_url_view
```

```

from utils.config import TRUSTED_SOURCES, TRUSTED_RSS_FEEDS

from utils.text_utils import domain_from_url

from utils.article_extractor import fetch_news_from_url

from utils.stance import stance_from_text

from utils.scoring import model_agreement_line

from utils.analysis_engine import analyze_news

# =====

# ----- PAGE CONFIG -----

# =====

st.set_page_config(page_title="Rumour Detection System", page_icon="logo.png", layout="wide")

apply_theme()

st.markdown("""

<style>

.zig-h span{

    text-decoration-line: underline;

    text-decoration-style: wavy;

    text-decoration-color: #6C7A52; /* dark of #D7E4C1 */

    text-underline-offset: 6px;

}

.zig-t span{

    text-decoration-line: underline;

    text-decoration-style: wavy;

    text-decoration-color: #2D5A7A; /* dark of #D6EFFF */

```

```

        text-underline-offset: 6px;

    }

    .zig-u span{

        text-decoration-line: underline;

        text-decoration-style: wavy;

        text-decoration-color: #B08B2D; /* dark of #FFFACD */

        text-underline-offset: 6px;

    }

</style>

""", unsafe_allow_html=True)

# =====

# ----- LOAD MODEL -----

# =====

@st.cache_resource

def load_model():

    model_local = pickle.load(open("model/model.pkl", "rb"))

    vectorizer_local = pickle.load(open("model/tfidf.pkl", "rb"))

    return model_local, vectorizer_local

try:

    model, vectorizer = load_model()

except:

    st.error("Model files not found.")

    st.stop()

```



```

# =====

# ----- SESSION STATE -----

# =====

st.session_state.setdefault("history", [])

st.session_state.setdefault("voice_text", "")

st.session_state.setdefault("user_input", "")

st.session_state.setdefault("headline_input", "")

st.session_state.setdefault("extracted_text", "")

st.session_state.setdefault("url_input", "")

st.session_state.setdefault("mode", None)

# =====

# ----- HELPERS -----

# =====

def get_voice_input():

    r = sr.Recognizer()

    try:

        with sr.Microphone() as source:

            st.info("🎧 Listening...")

            r.adjust_for_ambient_noise(source, duration=1)

            audio = r.listen(source, timeout=5, phrase_time_limit=8)

            return r.recognize_google(audio, language="en-IN")

    except:

        st.warning("Voice input failed.")

```

```

        return ""

def analyze_and_render(user_input: str):

    if not (user_input or "").strip():

        st.warning("Please enter text or extract article.")

        return

    with st.spinner("Analyzing..."):

        result = analyze_news(

            user_input=user_input,

            model=model,

            vectorizer=vectorizer,

            trusted_sources=TRUSTED_SOURCES,

            max_hits=10,

            per_domain_hits=5,

        )

    if not result.get("ok"):

        st.error("Analysis failed.")

        return

    render_analysis_result(

        result=result,

        render_bullets_bold=render_bullets_bold,

        stance_from_text=stance_from_text,

        domain_from_url=domain_from_url,

        trusted_rss_feeds=TRUSTED_RSS_FEEDS,

```

```

        model_agreement_line=model_agreement_line, )

# =====

# ----- UI -----

# =====

mode = render_home_selector()

if mode is not None:

    render_back_button()

if mode == "headline":

    render_headline_view(analyze_and_render=analyze_and_render)

elif mode == "text":

    render_text_view(get_voice_input=get_voice_input, analyze_and_render=analyze_and_render)

elif mode == "url":

    render_url_view(fetch_news_from_url=fetch_news_from_url,
analyze_and_render=analyze_and_render)

# HISTORY

render_history()

```

The page title, icon, and layout are established through `st.set\_page\_config`. Visual customizations such as zigzag underlines are inserted using HTML and CSS through `st.markdown`. This is a common Streamlit pattern when the built-in widgets are not visually expressive enough.

4.2 User interface package

The `UI` package is logically separated into small modules, which is a sign of maintainable design. `home.py` contains the landing page with two logos, a pill-shaped title area, and three selectable modes. It changes the page background colour according to the active mode, which

helps users maintain context. ``headline_view.py`` provides a single input field and action button for headline analysis. ``text_view.py`` supports large text entry and integrates voice input. ``url_view.py`` handles URL entry, article extraction, and article preview before analysis.

``result_view.py`` is the richest module in the package. It displays top metrics, verdict banners, evidence strength gauge, explainable bullet list, official site checks, trusted RSS matches, Google News links, an optional evidence table, and a caution block. It also pushes a simplified history record into session state. ``history_view.py`` converts that history into a Plotly bar chart and a textual list of prior searches. This structure cleanly separates navigation, input, output, and persistence concerns.

4.3 News search and evidence collection

``utils/news_search.py`` provides the retrieval backbone. ``search_trusted_rss`` loops over the configured feed URLs, parses entries using ``feedparser``, computes word overlap between the query and article title, and retains results whose overlap score is above a threshold.

News_search.py code

```
import feedparser

from urllib.parse import quote_plus

from utils.text_utils import normalize_text

from utils.config import TRUSTED_RSS_FEEDS, OFFICIAL_AUTHORITIES

def search_trusted_rss(query_keywords: str, max_hits: int = 10):

    q = normalize_text(query_keywords)

    q_words = set(q.split())

    results = []

    for source, feed_url in TRUSTED_RSS_FEEDS.items():

        try:

            feed = feedparser.parse(feed_url)
```

```

for entry in feed.entries[:30]:

    title = entry.get("title", "")

    link = entry.get("link", "")

    t = normalize_text(title)

    t_words = set(t.split())

    if not q_words:

        continue

    overlap = len(q_words.intersection(t_words))

    score = overlap / max(len(q_words), 1)

    if score >= 0.25:

        results.append({

            "source": source,

            "title": title,

            "link": link,

            "score": round(score * 100, 2),

            "publisher_url": link

        })

    except:

        continue

results.sort(key=lambda x: x["score"], reverse=True)

return results[:max_hits]

def google_news_rss_search(query: str, max_hits: int = 10):

```

```

q = (query or "").strip()

if not q:

    return []

rss_url = (

    "https://news.google.com/rss/search?q="

    + quote_plus(q)

    + "&hl=en-IN&gl=IN&ceid=IN:en"

)

out = []

try:

    feed = feedparser.parse(rss_url)

    for entry in feed.entries[:max_hits]:

        title = entry.get("title", "") or ""

        link = entry.get("link", "") or ""

        publisher_url = ""

        src = "Google News"

        try:

            if hasattr(entry, "source") and entry.source:

                src = getattr(entry.source, "title", "Google News") or "Google News"

                publisher_url = getattr(entry.source, "href", "") or ""

        except:

            pass

```

```

        out.append({
            "source": src,
            "title": title,
            "link": link,
            "score": 0,
            "publisher_url": publisher_url
        })
    except:
        return []

    return out

def detect_official_domains(query_text: str):
    t = (query_text or "").lower()

    domains = []

    for key, doms in OFFICIAL_AUTHORITIES.items():
        if key in t:
            domains.extend(doms)

    seen = set()

    out = []

    for d in domains:
        d = d.lower().replace("www.", "")

        if d not in seen:
            seen.add(d)

```

```

        out.append(d)

    return out

def search_official_sites_via_google_news(keywords: str, domains: list, per_domain_hits: int
= 5):

    hits = []

    kw = (keywords or "").strip()

    if not kw or not domains:

        return hits

    for d in domains:

        q = f'{kw} site:{d}'

        res = google_news_rss_search(q, max_hits=per_domain_hits)

        for it in res:

            hits.append({

                "domain": d,

                "title": it.get("title", ""),

                "link": it.get("link", ""),

                "source": it.get("source", "Official")

            })

    return hits

```

4.4 Stance detection heuristics

The stance detector is intentionally rule based. It checks for phrases such as “fact check,” “debunked,” “hoax,” “misleading,” “claim is false,” and “fake news” to mark contradiction. It also distinguishes contexts where the word “fake” refers to a fake object uncovered by an

authority, which may actually support the claim being described. If uncertain language such as “reportedly” or “allegedly” is present, the function can return `UNCLEAR`.

4.5 Training pipeline

The file `train\_model.py` reveals how the model artifacts were generated. It expects two CSV files: `data/Fake.csv` and `data/True.csv`. These are labelled as 0 and 1 respectively, concatenated, shuffled, and then converted into a `content` field by joining title and text when both are available.

Train_model.py code

```
import pandas as pd

import pickle

import os

from sklearn.model_selection import train_test_split

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.linear_model import LogisticRegression

from utils.text_utils import clean_text

# Create model folder if not exists

if not os.path.exists("model"):

    os.makedirs("model")

# Load dataset

fake = pd.read_csv("data/Fake.csv")

true = pd.read_csv("data/True.csv")

fake["label"] = 0

true["label"] = 1
```

```

df = pd.concat([fake, true])

df = df.sample(frac=1).reset_index(drop=True)

# Use title + text if available

if "title" in df.columns and "text" in df.columns:

    df["content"] = df["title"] + " " + df["text"]

elif "text" in df.columns:

    df["content"] = df["text"]

# Clean text

df["content"] = df["content"].apply(clean_text)

# Train test split

X_train, X_test, y_train, y_test = train_test_split(

    df["content"], df["label"], test_size=0.2, random_state=42

)

# TFIDF

vectorizer = TfidfVectorizer(max_features=5000)

X_train_vec = vectorizer.fit_transform(X_train)

# Model

model = LogisticRegression(max_iter=200)

model.fit(X_train_vec, y_train)

# Save

pickle.dump(model, open("model/model.pkl", "wb"))

pickle.dump(vectorizer, open("model/tfidf.pkl", "wb"))

```

```
print("Model trained and saved successfully.")
```

Because the training data files are not part of the uploaded ZIP archive, the report cannot claim any measured accuracy from actual execution of this script. However, the pipeline itself is complete and standard. It would be straightforward to reproduce if the required CSV files were available.

4.6 Dependency analysis

Library	Observed use in project
streamlit	Web interface, session state, forms, metrics, warnings, and layout.
pandas	History dataframe creation and possible data handling in training.
plotly	Gauge chart for evidence/model confidence and bar chart for history.
feedparser	RSS parsing for trusted publishers and Google News results.
newspaper3k	Article extraction from URLs.
scikit-learn	TF-IDF vectorizer and logistic regression classifier.
deep-translator	Translation of non-English input to English.
langdetect	Automatic language detection.
SpeechRecognition	Voice-based text capture through Google recognizer.
sqlite3	Database persistence via local SQLite files.

Table 4.1 Key Python dependencies and their roles

4.7 Security and robustness observations

A scan of the supplied source code did not reveal obvious malicious behaviour such as arbitrary shell execution, use of dangerous subprocess calls, or destructive file deletion. However, two important engineering risks should be documented. First, the model and vectorizer are loaded with ``pickle.load()``. Pickle is convenient for machine learning artifacts but unsafe when files are untrusted, because loading a malicious pickle can execute arbitrary code. Second, broad

`except:` blocks appear in multiple functions. While these prevent the application from crashing, they also hide the precise cause of errors and make debugging harder.

Network dependency is another robustness issue. RSS fetching, article extraction, translation, and speech recognition all rely on external services or internet connectivity. In an offline setting, many functions would degrade or fail. A production-ready version should use targeted exception handling, structured logs, fallback messages, and perhaps local caching of source results.

CHAPTER - 5

Testing, Observations and Discussion

The uploaded ZIP archive provides enough material to discuss functional testing, user-facing behaviour, and engineering quality even though full training data and benchmark logs are absent. This chapter therefore combines code-based observations with representative test scenarios. The aim is to show how the application is expected to behave under normal and abnormal conditions and to identify what parts of the system are mature versus incomplete.

5.1 Observed package details

- The model file is a scikit-learn LogisticRegression classifier with class labels [0, 1].
- The TF-IDF vectorizer contains a vocabulary of 5000 features, matching the training script configuration.
- The supplied SQLite file contains `users` and `history` tables, indicating some user tracking capability beyond the visible interface.
- The current Streamlit application stores recent searches in session state and visualises them in a Plotly bar chart.
- The package includes two logo images used by the landing page.
- The result view is intentionally evidence-first and repeatedly warns the user not to trust the model blindly.

5.2 Representative functional test cases

Test case	Input type	Expected behaviour	Observed/derived outcome
TC-1	Empty headline	System should reject analysis and show warning.	Implemented through `analyze_and_render` empty input check.
TC-2	Normal English headline	Model prediction should execute and evidence search should start.	Supported by headline mode and analyzer pipeline.
TC-3	Hindi or mixed-language text	Language should be detected and translated before prediction.	Handled by `_safe_detect_lang` and `_safe_translate_to_en`.

TC-4	Invalid or inaccessible URL	System should fail gracefully and show extraction error.	URL mode stores empty extraction and displays error/warning.
TC-5	Voice capture unavailable	System should show warning without crashing.	Handled by broad exception in <code>`get_voice_input`</code> .
TC-6	Authority-related claim mentioning RBI/UGC/WHO	Official domains should be detected and searched.	Supported through <code>`detect_official_domains`</code> and site-specific search.
TC-7	Repeated searches in one session	History chart and list should update.	Implemented in <code>`render_analysis_result`</code> and <code>`render_history`</code> .

Table 5.1 Representative functional test cases for the application

5.3 Strengths of the current implementation

The strongest aspect of the implementation is its hybrid design. The project does not stop at a classifier; it brings together machine learning, source verification, translation, and user guidance. This makes the software more educationally valuable than a simple sentiment demo. Another strength is modularity. Most functions are placed in purpose-specific modules, allowing a reader to understand the code systematically. The use of Streamlit caching for model loading is also a good optimization. Support for voice and URL extraction improves usability and demonstrates awareness of real-world input diversity.

An additional strength is transparency. The model signal is clearly labelled as secondary, and the application explains why the final result was chosen. This design reflects an understanding that decision-support systems should expose reasoning, not merely labels. For project evaluation in an academic setting, such transparency can be as important as raw algorithmic sophistication.

5.4 Limitations and gaps

Several limitations are visible from the scan. The first is naming inconsistency: the project title suggests rumour detection, but the implementation behaves like authenticity analysis. This may create confusion during viva or report evaluation unless clarified explicitly. The second limitation is the absence of training data and measured performance records in the ZIP package.

Without those, claims about accuracy, precision, recall, or F1-score cannot be responsibly reported. The third limitation is reliance on external services. Translation, RSS feeds, Google News RSS, article extraction, and speech recognition can all fail due to connectivity or service changes.

The code also uses broad exception handling, which masks operational details. Database usage appears partially inconsistent because ``database/db.py`` references one database path while the supplied package includes a different SQLite file. Finally, serialized pickle files, while common in educational projects, introduce a security risk if their provenance is uncertain.

5.5 Performance discussion without unsupported metrics

In the absence of benchmark logs, performance discussion must remain qualitative. The chosen ML architecture should be computationally light because TF-IDF vectorization and logistic regression are efficient on CPU hardware. Streamlit adds minimal deployment complexity. The main latency contributors are likely network-bound operations such as RSS retrieval, official-site search, article extraction, translation, and voice recognition. Thus, user-perceived response time will depend less on the model and more on internet services. In practice, this means the interface may feel fast for short text-only analysis but slower when evidence retrieval is performed extensively.

Because the result logic is evidence-first, the utility of the system does not depend only on classifier quality. Even a moderately accurate model can become more useful when combined with transparent external sources. However, the opposite is also true: during breaking news or when search results are sparse, the evidence layer may be inconclusive and the final result may remain mixed or cautious.

5.6 Deployment and execution steps

To run the system on a local machine, a Python virtual environment should be created first. After activation, ``pip install -r requirements.txt`` installs the necessary libraries. The application can then be launched with Streamlit using ``python -m streamlit run app.py`` or the equivalent environment-specific command. Before execution, the ``model`` directory should contain the required ``.pkl`` files. If retraining is desired, the ``data/Fake.csv`` and ``data/True.csv`` files must be provided in the expected location. Since URL extraction, RSS queries, translation, and voice

recognition require network access, an active internet connection is recommended during demonstration.

For academic submission, screenshots of the running system may be added later if desired. The present report focuses on the scanned package itself and therefore does not fabricate screenshots that were not produced from an actual run of the software.

CHAPTER - 6

Conclusion and Future Scope

The scanned project package presents a meaningful educational software artifact that combines natural language processing, machine learning, source retrieval, user interface design, and database concepts in one integrated Streamlit application. Despite being titled “Rumour Detection System,” the implementation behaves as a credibility and evidence aggregation tool for news-like content. That mismatch should be explained during submission, but it does not reduce the technical learning value of the project. In fact, the system demonstrates a mature design decision by treating the machine learning output as secondary and by foregrounding external evidence and official-source checks.

From a reporting perspective, the project is suitable for a B.Tech level submission because it shows clear modularization, practical use of libraries, and a deployable user interface. The package also reveals real-world engineering issues such as evolving database schemas, dependence on external services, and the security implications of serialized models. These issues enrich the academic discussion rather than weakening it. Overall, the project can be presented as a hybrid AI-enabled verification assistant whose core contribution lies in combining multiple weak signals into a transparent decision-support workflow.

6.1 Future scope

Area	Recommended enhancement	Benefit
Modeling	Replace or compare logistic regression with transformer-based classifiers.	Better semantic understanding and robustness on nuanced language.
Rumour focus	Introduce satire, sarcasm, and joke classification datasets.	Align implementation with the project title and broaden NLP value.
Explainability	Add keyword attribution, source weighting display, and confidence calibration.	Improves trust and academic interpretability.
Database	Unify schema and implement authenticated multi-user history storage.	Better consistency and real-user management.
Security	Replace pickle with safer model packaging or signed artifacts.	Reduces risk from untrusted serialized files.

Offline support	Cache feeds and translation results locally or add fallback paths.	Improves resilience during weak internet connectivity.
Evaluation	Record accuracy, confusion matrix, latency, and user-study feedback.	Strengthens scientific credibility of the report.

Table 6.1 Limitations converted into future enhancement opportunities

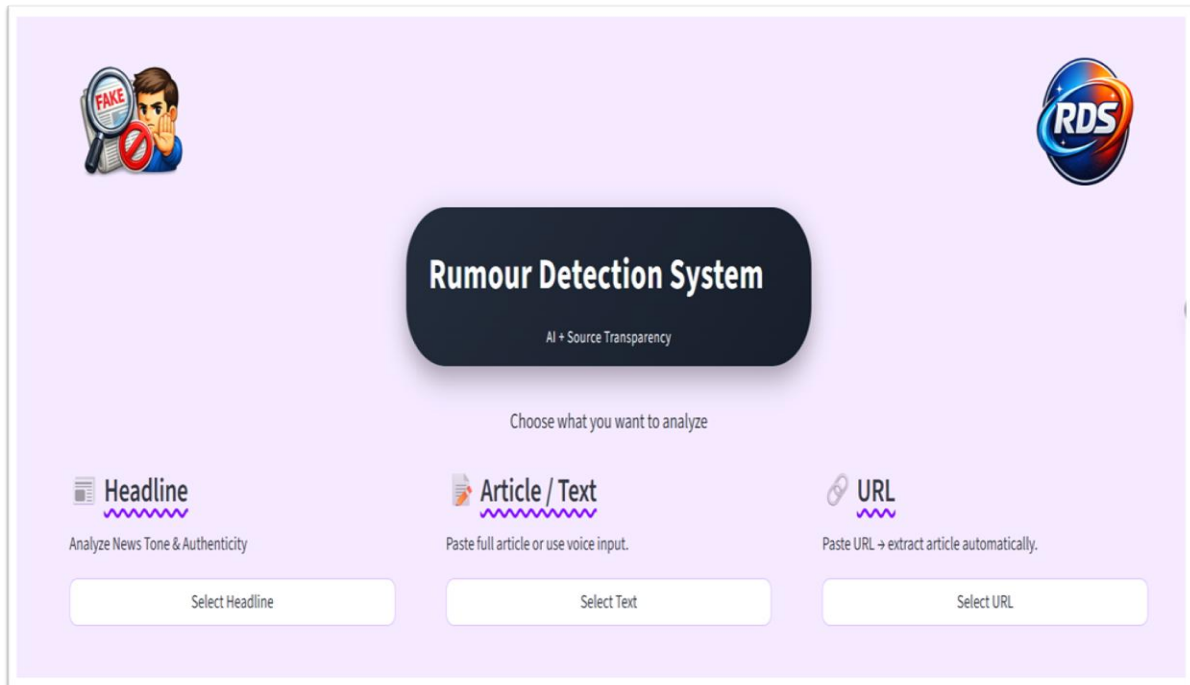
REFERENCES

- [1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing and Management*, vol. 24, no. 5, pp. 513-523, 1988.
- [2] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [4] Streamlit Documentation, "Build data applications in pure Python,".
- [5] Google. Search Engine
Available at: <https://www.google.com/>
- [6] Geeksforgeeks. Computer Science and Programming Articles.
Available at: <http://www.geeksforgeeks.org/>
- [7] AI-based Assistance Tool.
Available at: <https://chatgpt.com/>
- [8] Datasets and Machine Learning Resources.
Available at: <https://kaggle.com/>

APPENDIX

Appendix A: Screenshots of the application

Home Page



Headline Input Page



Text / Voice Input Page

Analyze by Article / Text

Enter Article Text to analyze (or paste full news)

Analyze News (Text)

Text Analysis Result Page

Analyze by Article / Text

Enter Article Text to analyze (or paste full news)

We are a Non-Government Entity provide jobs information gathered from various trusted sources. We also provide official source pdf link or website URL at the end of each job. All the content provided here is only for the summarized information purpose to the job aspirants. GovtJobGuru team does not call any person for job offers or job assistance. GovtJobGuru.in never charge any candidate or person for any job. Please beware of fake calls or emails. For any assistance, please contact us on email at admin@govtjobguru.in While all efforts have been made to make the Information available on this App/website as Authentic as possible. We are not responsible for any loss to anybody or anything caused by any Shortcoming, Defect or Inaccuracy of the Information on this Website. Please check Official Government Website twice before applying for any job.

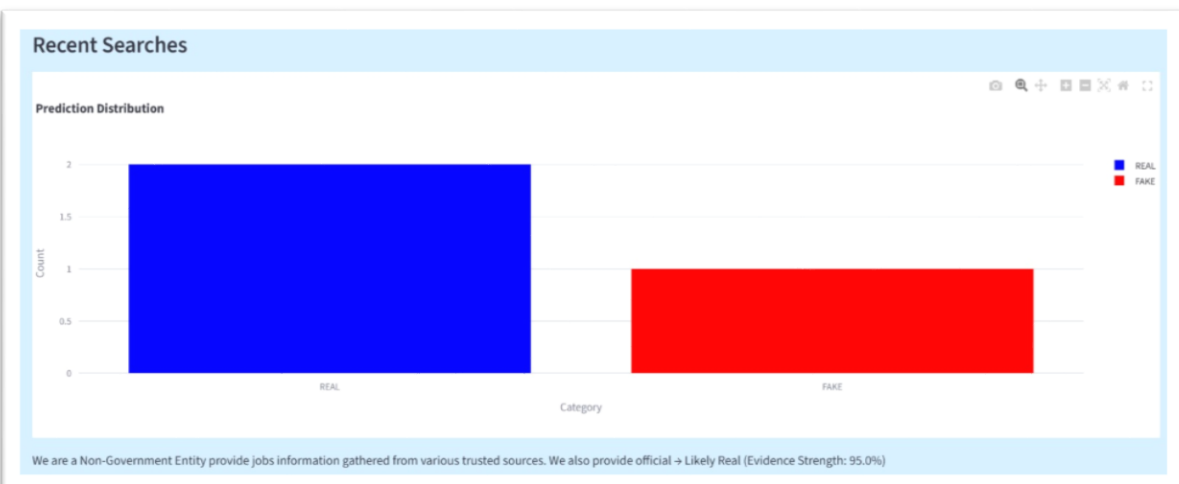
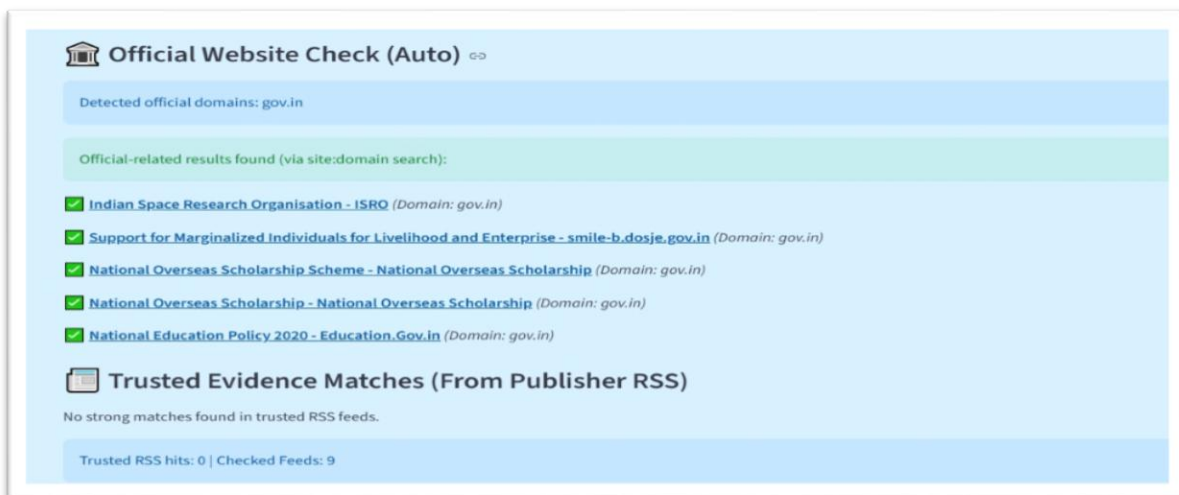
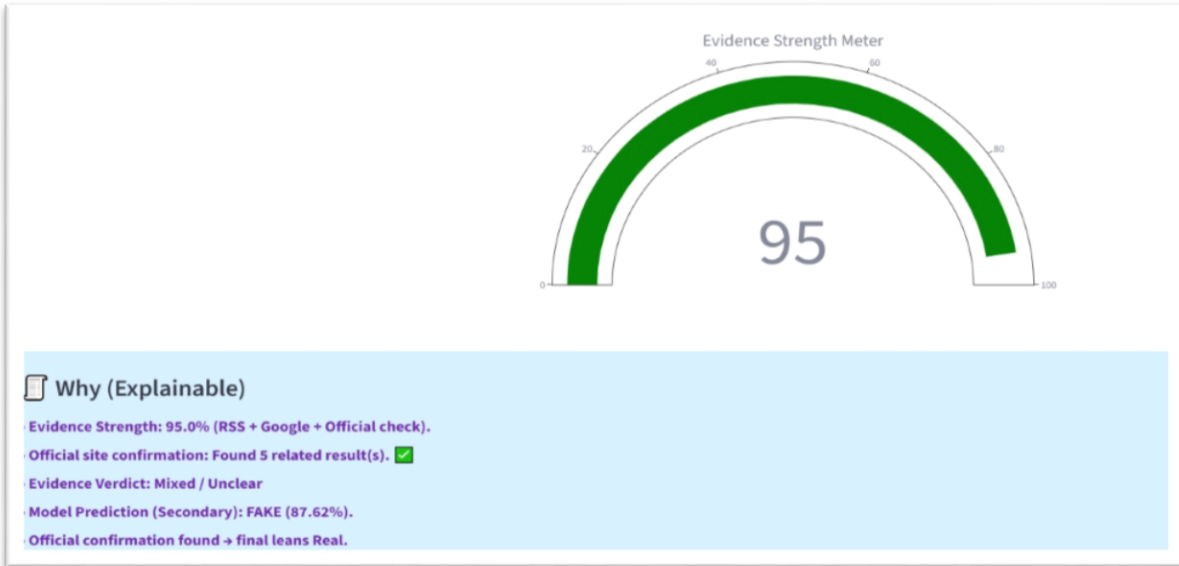
Analyze News (Text)

Final Analysis Result

Final Label	Evidence Strength	Official Check
Likely Real	95.0%	FOUND

FINAL: Likely Real

Evidence Visualization



URL Input Page

Analyze by URL (Extract Article Automatically)

Paste News URL to analyze

Extract Article

Analyze this Article (URL)

Recent Searches

No searches yet.

Article Extraction View

Analyze by URL (Extract Article Automatically)

Paste News URL to analyze

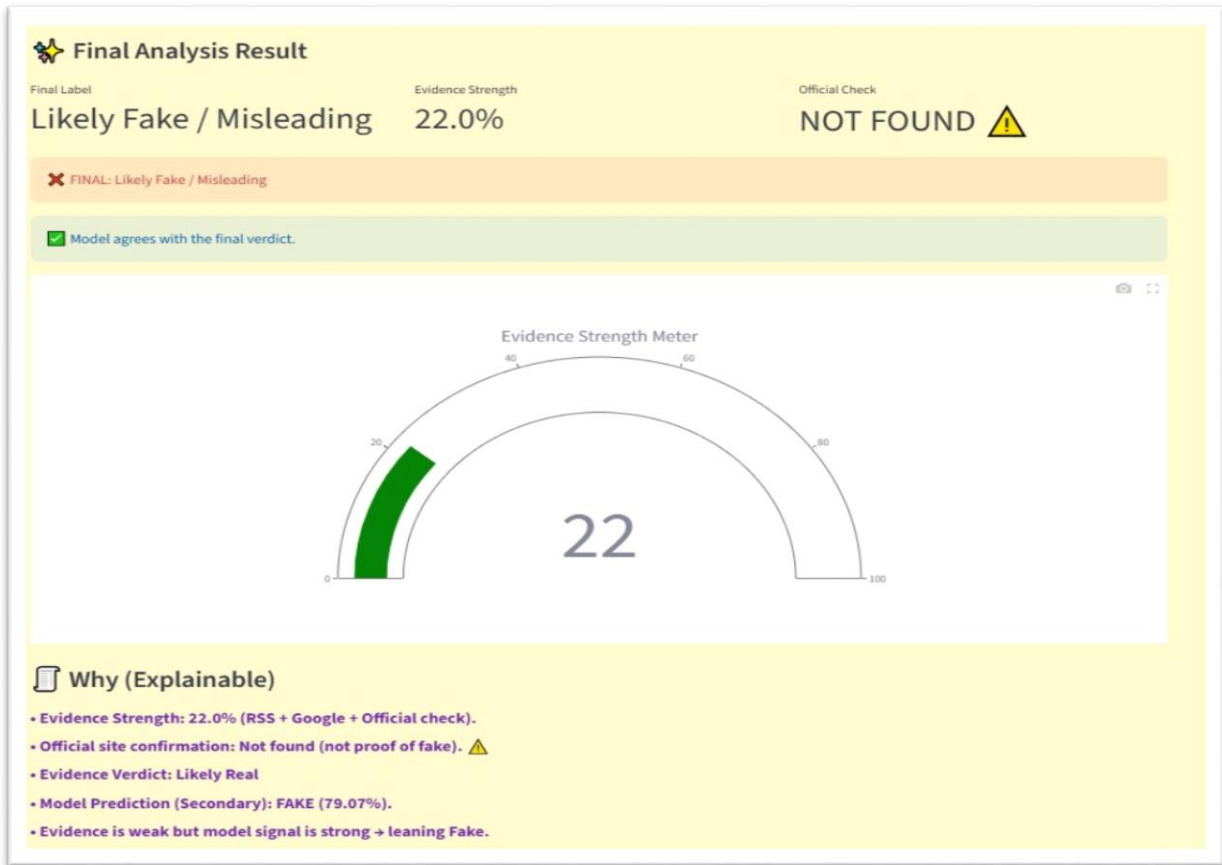
Extract Article

Analyze this Article (URL)

☒ Article extracted. Now analyze below (no copy/paste needed).

>  View Extracted Article (Click to expand)

URL Analysis Result Page



Appendix B: Execution Commands

1. Create Virtual Environment: `python -m venv .venv`
2. Activate Virtual Environment: `.venv\Scripts\activate`
3. Install Required Dependencies: `pip install -r requirements.txt`
4. Run the Application: `python -m streamlit run app.py`